

TizenRT

Memory Allocator Analysis

Weaknesses found, and every approach available for improving stability

Field	Value
Subject	The TinyAra heap allocator and the rest of os/mm
Question addressed	Which changes would most improve memory subsystem stability
Basis	Reading the sources; every claim carries a file and line reference
Sources reviewed	os/mm/mm_heap, os/mm/mm_gran, os/mm/kmm_heap, os/mm/umm_heap
Not done	No build, no run on hardware, no profiling. See section 8.

Contents

1. The Constraint That Governs Everything Else

2. What the Allocator Actually Is

3. Weaknesses Found, With Evidence

3.1 The anti-fragmentation feature in the tree is inert by default

3.2 Allocation and free are both $O(n)$ under a global lock

3.3 There is no overflow detection at all

3.4 Stacks are allocated from the same heap

3.5 The failure path dumps with interrupts disabled

3.6 Free-list validation is nearly a no-op

3.7 `MM_MIN_SHIFT` is pinned against its own derivation

4. Tier 0 - Configuration Only, No Code

5. Tier 1 - Detection, Small and Self-Contained

6. Tier 2 and 3 - Fragmentation and Allocator Algorithm

7. Tier 4 - Policy and Architecture

8. Recommended Order of Work

Appendix A. Evidence Index

1. The Constraint That Governs Everything Else

os/mm/Kconfig declares CONFIG_MM_ASSERT_ON_FAIL with **default y**, and mm_manage_allocfail.c:209 ends with:

```
#ifdef CONFIG_MM_ASSERT_ON_FAIL
    PANIC();
#endif
```

with WRITE_REBOOT_REASON(REBOOT_SYSTEM_MEMORYALLOCFAIL) above it. So on a default TizenRT build, **any failed allocation reboots the device**.

That reframes the whole stability question:

- Fragmentation is not a performance issue, it is a **reboot** issue. A heap with 300 KB free but no 4 KB contiguous block panics.
- A slow leak is a scheduled reboot.
- A transient allocation spike is a reboot.
- Anything that reduces the **largest free block** over time is a field failure mechanism.

The highest-value work is therefore not better failure handling. It is **never reaching failure**. Everything below is ordered by that.

2. What the Allocator Actually Is

A NuttX-derived best-fit segregated-list allocator:

Property	Detail
Size classes	19 buckets. $MM_NNODES = MM_MAX_SHIFT - MM_MIN_SHIFT + 1 = 22 - 4 + 1$
Bucket span	Each spans a power of two: bucket k holds chunks in $[32 \times 2^k, 32 \times 2^{(k+1)})$
Ordering	Free chunks within a bucket are kept sorted descending by size
Granule	16 bytes (MM_MIN_SHIFT forced to 4), immediate bidirectional coalescing on free
Locking	One recursive semaphore per heap; ISR-context frees deferred to a per-CPU delay list
Multi-heap	Optional via CONFIG_KMM_NHEAPS and CONFIG_KMM_REGIONS
Granule allocator	os/mm/mm_gran/ exists but has zero callers outside itself

Real heap sizes taken from the shipped defconfigs: 131072 (128 KB), 466944 (456 KB), 968704 (946 KB, the most common), and some DDR boards at 64 MB.

3. Weaknesses Found, With Evidence

3.1 The anti-fragmentation feature in the tree is inert by default

CONFIG_MM_ALLOC_LARGE_FROM_BACK separates large and small allocations to opposite ends of a free chunk, which is a genuinely good idea. But CONFIG_MM_LARGE_ALLOC_THRESHOLD **defaults to 1048576 (1 MB)**, and the majority of boards have heaps of 128 KB to 946 KB. **The threshold is larger than the entire heap. The feature can never fire.**

This is the cheapest available win in the whole subsystem: a defconfig change, no code.

3.2 Allocation and free are both O(n), unbounded, under a global lock

mm_addfreechunk.c:93 performs a sorted insertion:

```
for (prev = &heap->mm_nodelist[ndx], next = prev->flink;  
     next && next->size > node->size; prev = next, next = next->flink) ;
```

and mm_malloc.c:197 walks the same list for best fit. Both are linear in bucket length.

Because buckets are power-of-two wide, a workload with many similar-sized objects - 80-byte network buffers, 96-byte message structs - puts all of them in one bucket. On a 946 KB heap that can be thousands of nodes. Every free() then walks thousands of links with the memory manager semaphore held.

For a real-time operating system this is the wrong shape: free() should not have a worst case proportional to heap occupancy, and it should not hold a global lock while doing it.

3.3 There is no overflow detection at all

A search of os/mm for redzone or canary logic returns nothing. Concretely there is:

- no redzone or canary after the payload;
- no magic or checksum in struct mm_allocnode_s - it is just preceding, the optional heapinfo fields, and size;
- so a one-byte overflow past a chunk silently corrupts the next chunk's header.

Detection happens only when mm_check_heap_corruption() walks the whole heap comparing prev->size against MM_PREV_NODE_SIZE(node), and that runs **only from the assert and crash paths** (up_assert.c, arm_assert.c) and from heapinfo_parse_heap. So corruption is discovered at crash time, arbitrarily far from its cause, with no attribution to the culprit.

3.4 Stacks are allocated from the same heap

up_createstack.c:186-198 - every task stack comes from kmm_memalign() or kumm_memalign() on the general heap. Stacks are large, long-lived and alignment-constrained. Mixing them with short-lived small allocations is the textbook recipe for external fragmentation, and memalign additionally leaves an odd-sized remainder in front of each aligned block.

3.5 The failure path does a huge dump with interrupts disabled

`mm_manage_alloc_fail()` calls `enter_critical_section()` at line 195 and *then* dumps every heap, every application's heap, and a full per-PID heapinfo breakdown before `PANIC()`. On a multi-application build that is a very long time with interrupts off. It is a panic path so the bar is lower, but it risks the watchdog firing before the diagnostic reaches the console - losing exactly the information the device rebooted to collect.

3.6 Free-list validation is nearly a no-op

`DEBUGASSERT_MM_FREE_NODE` (`mm_node.h:35`) checks only `node->blink != NULL`. Any non-zero garbage in `blink`, and any corruption of `fblink`, passes.

3.7 MM_MIN_SHIFT is pinned against its own derivation

`mm.h:148` has the computed value commented out:

```
// #define MM_MIN_SHIFT (MM_MIN_SHIFT_BASE + MM_MIN_SHIFT_DEBUG +  
MM_MIN_SHIFT_64BIT)  
#define MM_MIN_SHIFT 4
```

with a note that 5 "should be tested". The header's own table says debug and 64-bit configurations want 5 or 6. The split guard saves this on 32-bit, but on 64-bit with `HEAPINFO`, `SIZEOF_MM_ALLOCNODE` is not a multiple of 8, so `mm_malloc`'s documented promise that "8-byte alignment of the allocated data is assured" does not hold.

4. Tier 0 - Configuration Only, No Code

Approach	Effect	Risk
Retune <code>MM_LARGE_ALLOC_THRESHOLD</code> to about 2 to 8 KB and enable <code>MM_ALLOC_LARGE_FROM_BACK</code>	Activates the existing large/small segregation; directly attacks fragmentation	Low
Use <code>KMM_NHEAPS</code> / <code>KMM_REGIONS</code> to give long-lived subsystems their own heap	Lifetime segregation without new code	Low
Enable <code>DEBUG_CHECK_FRAGMENTATION</code> in a soak build	Produces the fragmentation histogram needed to justify everything else	None

Start here. Most of the fragmentation headroom may be recoverable without writing a line of code.

5. Tier 1 - Detection, Small and Self-Contained

5a. Redzones

A 4-byte magic immediately after the user payload, written on allocation, verified on free and during the heap walk. Costs one granule on some allocations, often zero, since sizes are already rounded up to 16 and the slack is currently wasted. This is the single biggest detection gap, and it composes with poison-based use-after-free work rather than competing with it.

5b. Header integrity

Store `size ^ ~preceding`, or a small magic, in the allocnode and validate it on every `free()` and `realloc()`. Turns "the heap walk explodes later" into "this exact pointer's header is damaged, here is its owner".

5c. Periodic background check

Run `mm_check_heap_corruption()` from the idle task or a low-priority timer, region by region, releasing the semaphore between regions. Moves detection from crash time to within seconds of the cause. The function already retakes the semaphore per region, so it is close to ready for this.

5d. Tighten the free-node assertion

Bounds-check flink and blink against the heap region rather than only testing blink for NULL.

5e. Poison on free, and quarantine

Fill a freed chunk with a known pattern and verify it before reuse, and hold freed chunks out of circulation for a while so the check has a window in which to work. This is the only mechanism that can see a write through a stale pointer after the block has been reallocated to somebody else, because at that point the address legitimately belongs to the new owner and no content check can separate the two writers.

6. Tier 2 and 3 - Fragmentation and Allocator Algorithm

6.1 Tier 2 - fragmentation control, moderate effort

- **Slab or pool layer for hot sizes.** Front the general allocator with fixed-size pools for the two or three sizes the workload allocates most. Pool allocations never fragment the main heap and are $O(1)$. This is the highest-payoff structural change for a device that must run for months. The dormant `mm_gran` code is a possible starting point.
- **Move stacks off the general heap.** Give task stacks their own region or pool, removing the largest, longest-lived, alignment-constrained allocations from the pool that also serves 32-byte churn.

- **Best-fit tie-breaking by address.** With equal-sized candidates, prefer the lowest address. Concentrates long-lived allocations at one end and keeps the top of heap contiguous. A small change to the bucket search.
- **Deferred or batched coalescing.** Partly present already via the delay list; could be extended.

6.2 Tier 3 - allocator algorithm, large effort

- **TLSF-style two-level segregated fit.** Replaces the sorted list with a bitmap-indexed free list: $O(1)$ allocation and $O(1)$ free, worst case, with bounded fragmentation properties that are formally studied. This is the standard answer for hard real-time allocation and would fix section 3.2 outright. It is a well-understood, self-contained algorithm, but it replaces the core of the allocator and needs a serious test campaign.
- **Unsorted buckets with finer size classes.** A cheaper middle path: keep the current structure but make buckets narrower and stop sorting within a bucket. Insertion becomes $O(1)$ and search becomes near- $O(1)$ because a bucket holds near-identical sizes. Much smaller change than TLSF, most of the determinism benefit.
- **Per-CPU magazines** for SMP builds, to cut contention on the global semaphore.

7. Tier 4 - Policy and Architecture

- **Pre-OOM reclaim chain.** Before `PANIC()`, call a registered list of reclaim callbacks - drop caches, shrink buffers, free non-critical allocations - and retry. The equivalent of Linux shrinkers. Converts a class of reboots into a degradation.
- **Low-watermark notification.** Signal subsystems when the largest free block or total free falls below a threshold, so they shed load before the hard failure.
- **Emergency reserve.** Hold back a small pool that only critical paths may draw from, so a non-critical allocation failing does not take the device down.
- **Revisit MM_ASSERT_ON_FAIL per product.** Fail-fast is defensible for finding bugs in development. In the field it converts every recoverable shortage into a reboot. This is a product decision, but it should be a conscious one; right now it is a default.
- **Trend telemetry.** Record largest-free-block and total-free over uptime. Fragmentation-driven failure is highly predictable from the trend, so it can be caught in soak testing instead of in the field.

8. Recommended Order of Work

1. **Measure.** Enable `DEBUG_CHECK_FRAGMENTATION` and log largest-free-block over a long soak on a real workload. Everything else should be justified by that data.
2. **Fix the MM_LARGE_ALLOC_THRESHOLD default.** Near-zero risk, and the feature is currently dead on most boards.

3. **Add redzones.** Biggest detection gap, small patch.
4. **Add the periodic background heap check.** Turns crash-time forensics into near-real-time detection.
5. **Slab the two or three hottest allocation sizes,** guided by step 1.
6. **Then consider the allocator rewrite** (section 6.2), only if step 1 shows the $O(n)$ behaviour or fragmentation is actually biting.

Caveat on all of the above. This comes from reading the code, not from running it. No build, no hardware run, no profiling was done. The ordering assumes a fragmentation-dominated failure mode, which is the common one for long-running embedded devices - but step 1 exists precisely to confirm that before anyone commits effort to steps 5 and 6.

Appendix A. Evidence Index

Reference	What it establishes
os/mm/Kconfig	MM_ASSERT_ON_FAIL defaults to y; MM_LARGE_ALLOC_THRESHOLD defaults to 1048576
mm_manage_allocfail.c:195, 209	Critical section around the dump; PANIC() on allocation failure
mm_addfreechunk.c:93	$O(n)$ sorted insertion into the size-class list
mm_malloc.c:197	$O(n)$ walk of the same list for best fit
mm.h:148-159	MM_MIN_SHIFT pinned to 4; 19 size classes
mm.h:261-275	mm_allocnode_s has no magic or checksum field
mm_node.h:35	DEBUGASSERT_MM_FREE_NODE only tests blink for NULL
up_createstack.c:186-198	Task stacks allocated from the general heap via memalign
up_assert.c, arm_assert.c	mm_check_heap_corruption called only from crash paths
os/mm/mm_gran/	Granule allocator present with zero callers elsewhere in the tree
build/configs/*/*/defconfig	Real heap sizes: 128 KB, 456 KB, 946 KB, and larger DDR boards